Atomistic Frame Collection Specification

Unified Data Model, Readers, and Preprocessing for Trajectories and Independent Ensembles

1. Purpose

This document defines the normalized frame container used by `mdstats` and the reader/preprocessor contract that constructs it.

The central class is:

`AtomisticFrameCollection`

It represents a fixed-population set of atomistic configurations with one of two explicit relationships between frames:

`FrameSemantics.TRAJECTORY`
`FrameSemantics.ENSEMBLE`

A **trajectory** is a time-ordered structural evolution. An **ensemble** is a collection of independent frames. In this specification, *ensemble* includes random MD snapshots, clustered representatives, manually generated structures, and any frame set for which adjacency and time ordering must not be used.

The distinction is necessary because RDF and coordination are per-frame structural observables, whereas MSD and autocorrelation functions require physical temporal ordering.

2. Design principles

The implementation follows six rules:

1. **Frame semantics are explicit.** An analysis must not infer time ordering from the presence of several frames.
2. **Source formats disappear at the normalization boundary.** Downstream code operates on one data model.
3. **Atom identity is fixed across frames.** Every array index refers to the same atom in every frame.

4. **Coordinates are canonical for their semantics.** Trajectories are time-unwrapped; ensembles are wrapped independently.
5. **Missing data are absent, not fabricated.** In particular, ensemble velocities are `None`.
6. **Structural and temporal capabilities are checked centrally.** Analysis functions use descriptive guards rather than relying on informal metadata.

3. Mathematical conventions

3.1 Cell convention

The cell matrix uses ASE's row-vector convention:

$$\mathbf{H}_t = \begin{pmatrix} \mathbf{a}_t^\mathsf{T} \\ \mathbf{b}_t^\mathsf{T} \\ \mathbf{c}_t^\mathsf{T} \end{pmatrix}.$$

For a row-vector fractional coordinate $\mathbf{s}_{it}$,

$$\mathbf{r}_{it} = \mathbf{s}_{it}\mathbf{H}_t.$$

The cell may vary by frame, including changes in volume, shape, and orientation.

3.2 Coordinate meaning

The stored field is:

```
fractional_positions  # shape (T, N, 3)
```

Its interpretation depends on `frame_semantics`.

Trajectory

For a trajectory, periodic crossings are accumulated so that

$$\tilde{\mathbf{s}}_{i,t} = \tilde{\mathbf{s}}_{i,t-1} + \Delta\mathbf{s}_{i,t}^{\mathrm{MIC}},$$

where the minimum-image increment in periodic direction α is

$$\Delta s_{i,t,\alpha}^{\mathrm{MIC}} = \Delta s_{i,t,\alpha} - \mathrm{round}(\Delta s_{i,t,\alpha}).$$

The Cartesian laboratory-frame trajectory is

$$\tilde{\mathbf{r}}_{i,t} = \tilde{\mathbf{s}}_{i,t}\mathbf{H}_t.$$

This representation is appropriate for MSD and finite-difference velocity reconstruction.

Ensemble

For an independent ensemble, no displacement is inferred between frames. Each periodic fractional component is reduced independently:

$$s_{i,t,\alpha}^{\text{wrapped}} = s_{i,t,\alpha} - \lfloor s_{i,t,\alpha} \rfloor.$$

There is no physically meaningful distinction between image 0 and image 1 for an isolated sample. Independent wrapping prevents arbitrary source image flags from creating false inter-frame continuity.

3.3 Stress and pressure

Stress is stored as a symmetric tensile-positive tensor in $\text{eV}/\text{Å}^3$. Pressure is derived as

$$P_t = -\frac{1}{3} \operatorname{tr} \sigma_t.$$

A scalar source pressure may be stored when a full tensor is unavailable.

4. Public enumerations

```python
from enum import Enum


class FrameSemantics(str, Enum):
    TRAJECTORY = "trajectory"
    ENSEMBLE = "ensemble"
```

`TRAJECTORY` asserts all of the following:

- frame order is physically meaningful;
- adjacent frames belong to one structural evolution;
- physical times are available;
- coordinates are unwrapped across frames;
- a multi-frame collection contains velocities, native or reconstructed.

`ENSEMBLE` asserts:

- frames are independent samples;
- stored order is only an indexing convention;
- physical times and source steps are optional labels;
- coordinates are wrapped per frame;

- velocities are absent.

An ensemble need not be a rigorously weighted thermodynamic ensemble.

5. Public data structure

```python
@dataclass(slots=True)
class AtomisticFrameCollection:
    frame_semantics: FrameSemantics | str
    frame_ids: NDArray[np.int64]                      # (T,)

    atomic_numbers: NDArray[np.int32]                 # (N,)
    masses: NDArray[np.float64]                       # (N,), amu
    pbc: NDArray[np.bool_]                            # (3,)

    steps: NDArray[np.int64] | None                   # (T,)
    times: NDArray[np.float64] | None                 # (T,), ps

    cells: NDArray[np.float64]                        # (T, 3, 3), Å
    origins: NDArray[np.float64]                      # (T, 3), Å
    fractional_positions: NDArray[np.float64]         # (T, N, 3)

    velocities: NDArray[np.float64] | None            # (T, N, 3), Å/ps
    forces: NDArray[np.float64] | None                # (T, N, 3), eV/Å

    stresses: NDArray[np.float64] | None              # (T, 3, 3), eV/Å³
    scalar_pressures: NDArray[np.float64] | None      # (T,), eV/Å³
    temperatures: NDArray[np.float64] | None          # (T,), K
    potential_energies: NDArray[np.float64] | None    # (T,), eV
    kinetic_energies: NDArray[np.float64] | None      # (T,), eV
    total_energies: NDArray[np.float64] | None        # (T,), eV

    provenance: FrameCollectionProvenance
    metadata: dict[str, Any]
```

5.1 Frame identifiers

`frame_ids` are unique identifiers within the current collection. They are not physical timesteps. A subset normally receives compact IDs $0, 1, \ldots, T' - 1$, while parent frame IDs are recorded in metadata.

`steps` and `times` preserve source labels when available. For an ensemble, they remain descriptive metadata and do not authorize temporal analysis.

5.2 Fixed atom population

The dense shape $(T, N, 3)$ requires:

- constant atom count;
- constant atomic number at each canonical atom index;
- constant mass at each canonical atom index;
- constant PBC flags;
- a nonsingular cell in every frame.

Bonds, coordination numbers, ring occupancy, and local topology may change.

Collections with different compositions or atom counts require a future ragged or heterogeneous dataset abstraction.

6. Capability properties and guards

```
collection.is_trajectory
collection.is_ensemble
collection.is_single_frame
collection.has_time_axis
collection.has_velocities
collection.has_forces
collection.coordinates_are_time_unwrapped
```

Temporal analyses must call the relevant guards:

```
collection.require_trajectory("MSD")
collection.require_minimum_frames(2, "MSD")
times = collection.require_time_axis("MSD")
```

Velocity autocorrelation additionally requires:

```
velocities = collection.require_velocities("VACF")
```

Expected exceptions include:

```
TrajectoryRequiredError
InsufficientFramesError
MissingTimeAxisError
MissingVelocityError
```

RDF, coordination, bond geometry, and frame descriptors do not call `require_trajectory()`.

7. Public coordinate methods

```
def get_positions(frames=slice(None)) -> FloatArray:
    """Return Cartesian positions using each frame's cell."""
```

```
def get_wrapped_fractional_positions(frames=slice(None)) -> FloatArray:
    """Wrap periodic components into [0, 1)."""


def get_wrapped_positions(frames=slice(None)) -> FloatArray:
    """Return Cartesian positions wrapped into each instantaneous cell."""
```

For one selected frame,

$$\mathbf{R}_t = \mathbf{S}_t \mathbf{H}_t.$$

For several frames, the implementation uses

```
np.einsum("tni,tij->tnj", fractional, cells)
```

without storing a redundant Cartesian trajectory.

8. Frame selection and semantic conversion

```
subset = collection.select_frames(
    frame_indices,
    frame_semantics=None,
)
```

The default preserves the input semantics. A trajectory subset can be converted to an ensemble:

```
samples = trajectory.select_frames(
    [10, 87, 205, 901],
    frame_semantics="ensemble",
)
```

This operation:

1. selects all frame-dependent fields;
2. wraps every selected frame independently;
3. discards velocities;
4. changes coordinate provenance to independent-frame wrapping;
5. records parent frame IDs in metadata.

A convenience method is provided:

```
samples = trajectory.as_ensemble()
```

An ensemble cannot be converted back into a trajectory. Temporal continuity, periodic image history, and velocity data have already been discarded.

9. Public readers

9.1 VASP frame reader

```
def read_vasp_frames(
    filename: str,
    *,
    start: int | None = None,
    stop: int | None = None,
    stride: int = 1,
    timestep_fs: float | None = None,
    reconstruct_velocities: bool = True,
    frame_semantics: FrameSemantics | str = FrameSemantics.TRAJECTORY,
    strict: bool = True,
) -> AtomisticFrameCollection:
    ...
```

Supported sources:

- `vasprun.xml`;
- `XDATCAR` and filenames ending in `XDATCAR`.

For trajectory semantics, `POTIM` or `timestep_fs` is required. For ensemble semantics, no time axis is required.

`vasprun.xml` may supply forces, stress, energies, temperatures, and native velocity arrays. `XDATCAR` supplies structures only.

9.2 LAMMPS custom-dump reader

```
def read_lammps_frames(
    dump_file: str,
    *,
    log_file: str | None = None,
    units: str | None = None,
    timestep: float | None = None,
    type_map: dict[int, str | int] | None = None,
    mass_map: dict[int, float] | None = None,
    start: int | None = None,
    stop: int | None = None,
    stride: int = 1,
    reconstruct_velocities: bool = True,
    frame_semantics: FrameSemantics | str = FrameSemantics.TRAJECTORY,
    strict: bool = True,
) -> AtomisticFrameCollection:
    ...
```

The reader supports orthogonal, restricted-triclinic, and general-triclinic boxes. Persistent `id` fields are required. Every frame is sorted by ID before all per-atom

arrays are stored.

Coordinate priority is:

```
xsu ysu zsu
xu  yu  zu
xs  ys  zs + image flags
x   y   z  + image flags
wrapped coordinates with inferred images
```

For an ensemble, all forms are converted independently to wrapped fractional coordinates; image continuity is not used.

9.3 Static structure reader

```
def read_structure(
    filename: str | Path,
    *,
    format: str | None = None,
    index: int = 0,
    ...,
) -> AtomisticFrameCollection:
    ...
```

The result is a one-frame ensemble. ASE-supported formats include POSCAR, CONTCAR, CIF, and LAMMPS data files.

9.4 Multi-file structure collection

```
def read_structure_collection(
    filenames: Sequence[str | Path],
    *,
    format: str | None = None,
    indices: int | Sequence[int] = 0,
    strict: bool = True,
    ...,
) -> AtomisticFrameCollection:
    ...
```

One structure is read from each file. The output is an independent ensemble. All selected structures must satisfy the fixed-population constraint.

10. Normalization pipeline

```
source reader
    |
    v
RawFrameCollection
```

```
    |
    +-- normalize units
    +-- reconstruct cells and origins
    +-- sort atoms by persistent IDs
    +-- verify species and masses
    |
    +-- trajectory ------------------------------+
    |      unwrap periodic coordinates           |
    |      establish physical times              |
    |      read or reconstruct velocities        |
    |                                            |
    +-- ensemble -------------------------------+
           wrap each frame independently
           discard velocities
           no physical time required
    |
    v
AtomisticFrameCollection
    |
    v
structural or temporal analysis guards
```

10.1 Canonical atom order

For source IDs $I_{t,j}$, each frame is sorted:

$$\pi_t = \mathrm{argsort}(I_t).$$

The sorted ID vector must be identical in every frame:

$$I_{t,\pi_t} = I_{0,\pi_0}.$$

The same permutation is applied to positions, species, masses, velocities, forces, image flags, and source type IDs. IDs are discarded after validation.

10.2 Trajectory velocity reconstruction

When native velocities are absent, Cartesian positions are first constructed:

$$\mathbf{r}_i(t_n) = \tilde{\mathbf{s}}_i(t_n)\mathbf{H}(t_n).$$

For an interior frame,

$$\mathbf{v}_i(t_n) \approx \frac{\mathbf{r}_i(t_{n+1}) - \mathbf{r}_i(t_{n-1})}{t_{n+1} - t_{n-1}}.$$

Second-order one-sided differences are used at endpoints when enough frames are available. Variable-cell affine motion is included because the derivative is taken after conversion to Cartesian coordinates.

Finite-difference velocity is suitable for transport diagnostics but can distort high-frequency spectra.

11. Input constraints

11.1 All collections

Required:

- at least one frame and one atom;
- finite coordinates, cells, masses, and origins;
- positive atomic numbers and masses;
- unique `frame_ids`;
- nonsingular cells;
- complete optional fields when present;
- symmetric stress tensors;
- fixed atom mapping and PBC.

11.2 Trajectory-specific

Required:

- physical `times`;
- strictly increasing times;
- strictly increasing source steps when stored;
- complete velocities for more than one frame;
- coordinates unwrapped across time.

11.3 Ensemble-specific

Required:

- `velocities is None`;
- independent per-frame wrapping;
- no assumption that `steps`, `times`, or frame order are monotonic.

12. Analysis compatibility

Analysis	Trajectory	Ensemble	Single frame
Pair RDF	Yes	Yes	Yes
Coordination distribution	Yes	Yes	Yes
Bond-distance distribution	Yes	Yes	Yes

Analysis	Trajectory	Ensemble	Single frame
Bond-angle distribution	Yes	Yes	Yes
Frame descriptors	Yes	Yes	Yes
Clustering	Yes, usually converted to ensemble	Yes	Degenerate
Rare-event filtering	Yes	Yes	Yes
MSD	Yes	No	No
VACF / velocity spectrum	Yes	No	No
Time autocorrelation	Yes	No	No

Structural calculations may average over frames in either semantic mode. For an ensemble, this is an average over independent samples rather than time origins.

Frame semantics also constrain stateful neighbor acceleration. Stateless dense and cell-list searches are valid for every collection. The high-level periodic neighbor policy may activate a Verlet cache automatically only for a multi-frame trajectory, because trajectory semantics certify meaningful frame ordering and continuous fractional coordinates. Independent ensembles remain stateless under `cache_mode="auto"`; explicit `cache_mode="verlet"` is an expert performance override and does not change scientific results.

13. Clustering and active-learning readiness

The fixed-shape collection is designed to support future functions such as:

```
labels = cluster_frames(collection, descriptor=...)
rare = select_frames_by_condition(collection, predicate=...)
representatives = select_cluster_representatives(collection, labels)
```

Candidate descriptors include:

- coordination-number histograms;
- rare coordination-state indicators;
- ring-site occupancy vectors;
- RDF or angular fingerprints;
- local-environment embeddings;
- model uncertainty or extrapolation scores.

Clustering operates frame by frame and therefore does not require trajectory semantics. A trajectory should normally be converted to an ensemble before selecting unordered representatives.

Parent frame IDs and source provenance must be retained so selected structures can be traced back to the original MD or active-learning run.

14. Edge cases and warnings

14.1 Sparse trajectory output

Minimum-image unwrapping is ambiguous if an atom moves by one-half or more of a periodic cell span between saved frames. Prefer explicit unwrapped coordinates or image flags, or save frames more frequently.

14.2 Random MD frames read as a trajectory

Randomly selected or reordered frames must use `frame_semantics="ensemble"`. Treating them as a trajectory can create false periodic crossings, meaningless velocities, and invalid autocorrelations.

14.3 Ensemble source times

An ensemble may retain original MD steps or times for traceability. Their presence does not make temporal analysis valid. `has_time_axis` is true only for trajectory semantics.

14.4 Native velocities in independent samples

Velocities are deliberately discarded when constructing an ensemble. This prevents accidental VACF or kinetic propagation across unrelated frames. If a future application needs a velocity-distribution dataset, it should use a separate explicitly named abstraction.

14.5 Different compositions

A training-data pool containing structures with different atom counts or compositions cannot be represented by one `AtomisticFrameCollection`. Split it into compatible collections or introduce a future heterogeneous dataset type.

14.6 Nonperiodic clusters

The current representation requires a full-rank 3×3 cell. Assign a simulation box before importing a cluster without one.

15. Minimal reproduction algorithm

```
function normalize(raw, semantics):
    sort every frame by persistent atom ID
    verify identical ID, species, mass, and PBC mappings

    if semantics == trajectory:
        require physical times
```

```
        convert source coordinates to wrapped fractional coordinates
        add explicit image flags when available
        otherwise infer minimum-image frame increments
        accumulate continuous fractional positions

        if native velocities are complete:
            retain them
        else if more than one frame:
            reconstruct velocities from Cartesian positions and times
        else:
            velocities = None

    else if semantics == ensemble:
        convert every frame independently to fractional coordinates
        wrap periodic components into [0, 1)
        velocities = None

    normalize optional forces, stress, energies, and temperature
    construct provenance
    validate all shapes and semantic invariants
    return AtomisticFrameCollection
```

This algorithm is sufficient to reproduce the implemented normalization behavior.